

UNIVERSIDAD SANTIAGO DE CALI

Facultad de Ingeniería · Ingeniería de Sistemas

DesposteApp

Sistema de Optimización de Precios para el Desposte de Ganado
Bovino

Documentación Técnica del Sistema

Proyecto	DesposteApp — Prototipo de software
Versión del documento	2.0
Versión del sistema	2.0.0
Fecha	11/06/2026
Estado	Prototipo funcional (entorno académico)
Autores	Juan Esteban Montilla Rayo · Juan Pablo Rodríguez Becerra · Rafael Ángel Davalos Villegas
Audiencia	Cliente técnico / equipo de desarrollo
Confidencialidad	Documento de uso interno

Tabla de Contenido

Para actualizar la numeración de páginas y las entradas, haga clic derecho sobre la tabla y seleccione «Actualizar campos» (o presione F9) en Microsoft Word.

1. Introducción

1.1 Propósito del documento

Este documento describe el diseño, la arquitectura, los componentes y los requisitos de DesposteApp en su versión actual. Está pensado como referencia para quien deba evaluar, mantener o extender el sistema: explica cómo está construido, cómo se calculan los precios, cómo se integra con la fuente de datos del DANE y cómo se instala y opera. También dedica una sección a dejar por escrito lo que el sistema no hace, porque conocer los límites de un prototipo es tan importante como conocer sus funciones.

Esta es la segunda versión del documento. Reemplaza a la anterior e incorpora los cambios más relevantes del sistema desde entonces: el modelo de costeo por actividades (ABC) para distribuir los costos entre cortes, la simplificación del módulo SIPSA (los datos de mercado se consultan en vivo y ya no se almacenan), la reducción del modelo de datos a cinco tablas y el restablecimiento de contraseñas por parte del administrador.

1.2 Alcance del sistema

DesposteApp es un prototipo de software que apoya la toma de decisiones de precios en el proceso de desposte de ganado bovino. El sistema calcula precios de venta sugeridos por corte a partir de tres insumos: (1) el costo de compra del animal y los costos del proceso, distribuidos entre los cortes mediante costeo ABC; (2) los precios de referencia del mercado mayorista publicados por el DANE a través del servicio SIPSA; y (3) un margen de ganancia objetivo configurable. Además gestiona el registro de animales, cortes y costos, ofrece un panel de indicadores y administra usuarios con control de acceso por roles.

1.3 Audiencia

- **Cliente técnico:** Responsable de evaluar técnicamente la solución antes de su adopción.
- **Equipo de desarrollo:** Personas encargadas de mantener, corregir o ampliar el sistema.
- **Operación:** Personal que despliega y administra el entorno donde corre la aplicación.

1.4 Glosario y definiciones

Término	Definición
Desposte	Proceso de despiece de la canal del animal en cortes comerciales individuales.
Canal	Cuerpo del animal sacrificado, sin vísceras, piel ni extremidades, listo para despiece.
Peso vivo	Peso del animal en pie, antes del sacrificio.
Peso canal	Peso del animal una vez faenado (la canal).

Rendimiento en canal	Relación porcentual entre el peso canal y el peso vivo.
Merma	Pérdida de peso entre la canal y la suma de cortes vendibles (huesos, recortes, etc.).
Corte	Pieza comercial obtenida del despiece (p. ej. lomo fino, cadera, costilla).
Costeo ABC	Costeo basado en actividades: asigna los costos a los productos según los recursos que cada uno consume.
Inductor de costo	Criterio con el que un costo se reparte entre los cortes (peso, horas-hombre, kWh, etc.).
Factor de complejidad	Ponderador que refleja cuánto trabajo de proceso exige un corte respecto al promedio.
SIPSA	Sistema de Información de Precios y Abastecimiento del Sector Agropecuario (DANE).
DANE	Departamento Administrativo Nacional de Estadística de Colombia.
Margen de ganancia	Porcentaje de utilidad objetivo sobre el precio de venta.
JWT	JSON Web Token; estándar para tokens de autenticación firmados.
ORM	Object-Relational Mapping; mapeo objeto-relacional (SQLAlchemy).
SOAP	Protocolo de intercambio de mensajes basado en XML usado por el servicio del DANE.
SPA	Single Page Application; aplicación web de una sola página.

2. Descripción General del Sistema

2.1 Contexto y problema

En las plantas de desposte y comercializadoras de carne bovina, la fijación de precios suele realizarse de forma manual y empírica. Esto dificulta incorporar de manera consistente el costo real del proceso, el rendimiento en canal de cada animal y las variaciones del mercado mayorista. Un problema adicional, menos evidente, es que no todos los cortes cuestan lo mismo de producir: un lomo fino exige más trabajo de limpieza y porcionado que la carne molida, pero un reparto de costos por simple peso ignora esa diferencia. DesposteApp aborda ambos problemas: sistematiza el cálculo integrando costos, rendimiento y precios de referencia oficiales, y distribuye los costos entre cortes según la intensidad de proceso de cada uno.

2.2 Objetivos del sistema

- Centralizar el registro de animales, cortes y costos del proceso de desposte.
- Calcular el costo real por corte mediante costeo basado en actividades (ABC).
- Incorporar precios de referencia del mercado (DANE-SIPSA) al cálculo de precios.
- Sugerir precios de venta por corte aplicando un margen objetivo configurable.
- Ofrecer un panel de indicadores para apoyar la toma de decisiones.
- Controlar el acceso mediante autenticación y roles de usuario.

2.3 Características principales

Módulo	Descripción
Autenticación	Inicio de sesión con JWT en cookie HttpOnly, gestión de perfil y cambio de contraseña.
Animales	Registro y gestión de animales con peso vivo, peso canal, calidad y precio de compra.
Cortes	Gestión de cortes por animal, con categoría, peso, factor de complejidad y precios calculados.
Costos	Registro de costos por animal o globales, cada uno con su inductor de costo ABC.
SIPSA	Consulta en vivo de precios de referencia del mercado bovino del DANE, con promedios y tendencia por corte.
Análisis	Panel de indicadores, costo por kilogramo y cálculo de precios sugeridos por animal.
Usuarios	Administración de usuarios, roles y restablecimiento de contraseñas (solo perfil

	ADMIN).
--	---------

2.4 Roles de usuario

Rol	Permisos
ADMIN	Acceso completo a todos los módulos, incluida la administración de usuarios.
ESTANDAR	Acceso a operación (animales, cortes, costos, SIPSA, análisis); sin gestión de usuarios.

Usuarios iniciales

Al arrancar por primera vez, el backend crea automáticamente los usuarios definidos en las variables de entorno (un administrador y un operador). Las credenciales no están escritas en el código fuente: se leen de la configuración del entorno, y si no se define la contraseña de un usuario inicial, ese usuario no se crea.

2.5 Flujo de trabajo de extremo a extremo

El uso típico del sistema sigue una secuencia que va desde la compra del animal hasta el precio sugerido de cada corte. El recorrido completo, tal como lo vive el usuario, es el siguiente:

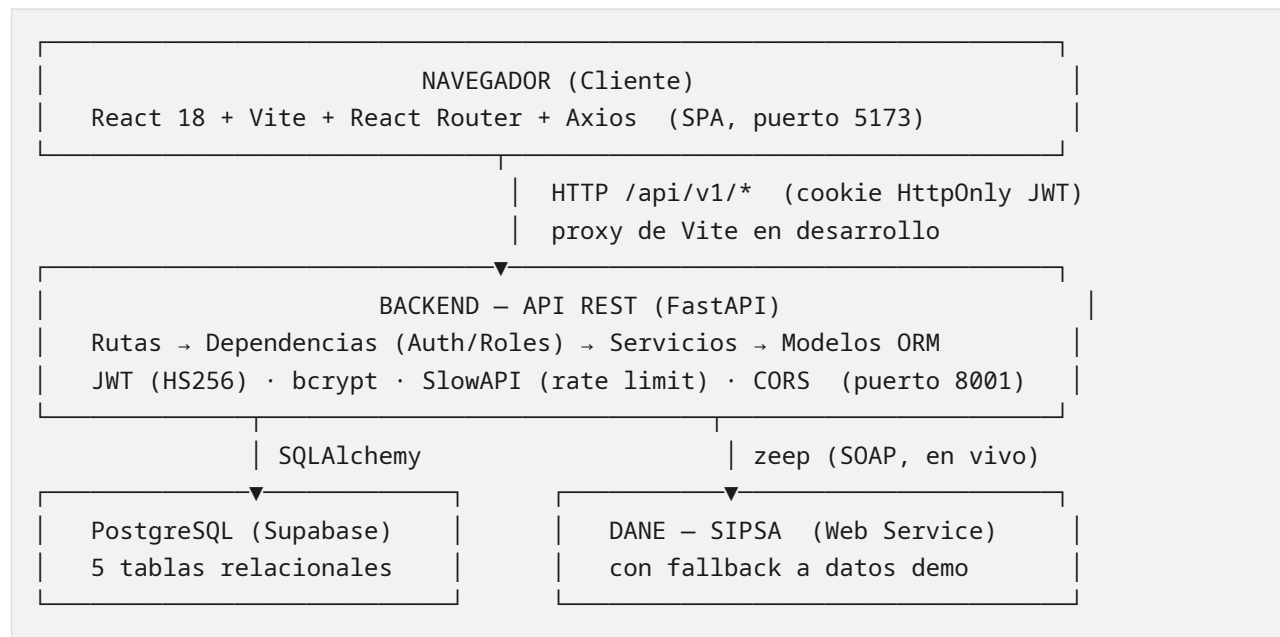
1. Inicio de sesión: el usuario se autentica en /login; el backend emite la cookie de sesión.
2. Registro del animal (módulo Animales): se captura el código, tipo, peso vivo, peso canal, calidad y precio de compra. El sistema calcula el rendimiento en canal si dispone de ambos pesos.
3. Registro de cortes (módulo Cortes): se registran los cortes obtenidos del despiece con su peso. El sistema les asigna automáticamente un factor de complejidad según el nombre del corte.
4. Registro de costos (módulo Costos): se ingresan los costos del proceso (transporte, sacrificio, energía, mano de obra...), asociados al animal o al fondo global, indicando el inductor con el que cada costo debe repartirse entre los cortes.
5. Consulta de mercado (módulo SIPSA): la aplicación consulta en vivo los precios mayoristas del DANE y muestra promedios, rangos y tendencia por corte. Esta consulta es informativa; el cálculo de precios la realiza de nuevo por su cuenta.
6. Cálculo de precios (módulo Análisis): el usuario selecciona el animal y el margen objetivo. El backend distribuye los costos entre los cortes (costeo ABC), mezcla el resultado con la referencia de mercado y devuelve el precio sugerido, el mínimo viable y el margen real por corte.

7. Persistencia de resultados: cada cálculo actualiza los campos del corte y deja un registro en la tabla de precios, de modo que queda trazabilidad de cómo se determinó cada precio.
8. Seguimiento (Dashboard): el panel muestra los totales, el costo promedio por kilogramo, el margen promedio y la tabla de referencia SIPSA.

3. Arquitectura del Sistema

3.1 Visión general

DesposteApp adopta una arquitectura cliente-servidor de tres capas, organizada como un monorepo con dos aplicaciones independientes: un frontend SPA (Single Page Application) en React y un backend de API REST en FastAPI. La persistencia se realiza sobre una base de datos PostgreSQL (en la nube, mediante Supabase). El backend consume además un servicio externo SOAP del DANE para obtener precios de referencia del mercado; estos datos se procesan en memoria en cada consulta y no se almacenan en la base de datos.



3.2 Estructura monorepo

El repositorio agrupa las dos aplicaciones bajo una misma raíz:

```
tesis/
├── backend/           API REST en FastAPI (Python)
│   ├── main.py       Punto de entrada y configuración de la aplicación
│   ├── requirements.txt
│   └── app/
│       ├── routes/    Routers FastAPI (capa de presentación)
│       ├── services/  Lógica de negocio (auth, análisis de precios)
│       ├── models/    Modelos ORM SQLAlchemy (tablas)
│       ├── schemas/   Esquemas Pydantic (validación E/S)
│       ├── dependencies/ Dependencias de autenticación y autorización
│       ├── database/  Motor, sesión y Base declarativa
│       └── sipsa/     Cliente SOAP DANE y procesamiento de datos
```

	└─ limiter.py	Limitador de peticiones (SlowAPI)
	└─ frontend/	SPA en React + Vite (JavaScript)
	└─ vite.config.js	Configuración y proxy /api
	└─ src/	
	└─ pages/	Una página por ruta
	└─ components/	Navbar, Toast, rutas protegidas, tarjetas KPI
	└─ context/	AuthContext (estado de sesión)
	└─ services/	api.js (instancia Axios)
	└─ App.jsx	Definición de rutas

3.3 Arquitectura por capas del backend

El backend separa responsabilidades en capas bien definidas:

Capa	Responsabilidad	Ubicación
Presentación (Routers)	Define los endpoints REST, valida entradas y delega en servicios.	app/routes/
Validación (Schemas)	Esquemas Pydantic para validar y serializar peticiones y respuestas.	app/schemas/
Dependencias	Autenticación, extracción de token y control de roles.	app/dependencies/
Negocio (Services)	Reglas de cálculo de precios, costeo ABC y autenticación.	app/services/
Datos (Models)	Modelos ORM SQLAlchemy mapeados a tablas.	app/models/
Integración	Cliente SOAP del DANE y procesamiento de datos de mercado.	app/sipsa/

3.4 Comunicación frontend-backend

- **Proxy:** En desarrollo, Vite expone un proxy que redirige toda petición a /api hacia el backend (http://localhost:8001), evitando problemas de CORS.
- **Cliente HTTP:** Axios se configura con baseURL /api/v1 y withCredentials: true, de modo que la cookie de sesión viaja automáticamente en cada petición.
- **Sesión:** El token JWT se almacena en una cookie HttpOnly emitida por el backend; el JavaScript del navegador no necesita (ni puede) leerlo, lo que reduce el riesgo de robo por XSS.

- **Manejo de sesión expirada:** Ante una respuesta 401, el interceptor de Axios limpia el estado local y redirige a /login.

3.5 Ciclo de vida de una petición

1. El usuario interactúa con una página React, que invoca un método del servicio Axios (api.js).
2. La petición sale hacia /api/v1/... con la cookie de sesión; el proxy de Vite la reenvía al backend.
3. FastAPI resuelve el router correspondiente; la dependencia de autenticación valida el JWT y el rol.
4. El router invoca la capa de servicio, que ejecuta la lógica de negocio usando los modelos ORM.
5. SQLAlchemy traduce las operaciones a SQL contra PostgreSQL y devuelve los resultados.
6. La respuesta se serializa con un esquema Pydantic y regresa al frontend en formato JSON.

4. Stack Tecnológico y Requisitos

4.1 Backend

Tecnología	Versión de referencia	Función
Python	3.11+	Lenguaje del backend.
FastAPI	0.111	Framework web/API REST.
Uvicorn	0.30	Servidor ASGI de ejecución.
SQLAlchemy	2.0	ORM y acceso a base de datos.
Pydantic	2.7	Validación y serialización de datos.
psycopg2-binary	2.9	Driver de PostgreSQL.
python-jose	3.3	Firma y verificación de JWT (HS256).
bcrypt	4.1	Hash de contraseñas.
zeep	4.2	Cliente SOAP para el servicio del DANE.
pandas / numpy	2.2 / 1.26	Procesamiento y limpieza de datos de mercado.
slowapi	0.1.9	Limitación de tasa de peticiones (rate limiting).
python-dotenv	1.0	Carga de variables de entorno desde .env.

Nota sobre versiones

Las versiones indicadas corresponden a las fijadas en requirements.txt. En entornos con Python muy reciente (p. ej. 3.13/3.14) puede ser necesario emplear versiones más actuales de numpy y pandas, que disponen de paquetes precompilados (wheels) compatibles.

4.2 Frontend

Tecnología	Versión de referencia	Función
Node.js	18+	Entorno de ejecución para construir el frontend.
React	18.3	Librería de interfaz de usuario.

Vite	5.2	Servidor de desarrollo y empaquetador.
React Router	6.23	Enrutamiento de la SPA.
Axios	1.7	Cliente HTTP hacia la API.
lucide-react	1.17	Iconografía SVG de la interfaz.

4.3 Base de datos

Motor relacional PostgreSQL. El prototipo está configurado para usar Supabase (PostgreSQL gestionado en la nube) a través de su «Session Pooler». El backend crea automáticamente el esquema (las cinco tablas) en el arranque y aplica de forma idempotente las columnas del modelo ABC sobre instalaciones previas, sin necesidad de migraciones manuales.

4.4 Servicios externos

El sistema consume el servicio web SIPSA del DANE (SOAP) para obtener precios de referencia del mercado bovino. Cuando el servicio no responde, el sistema recurre automáticamente a un conjunto de datos de demostración con precios realistas, de modo que la aplicación sigue siendo funcional. Los datos de mercado se consultan y procesan en cada petición; no se persisten localmente.

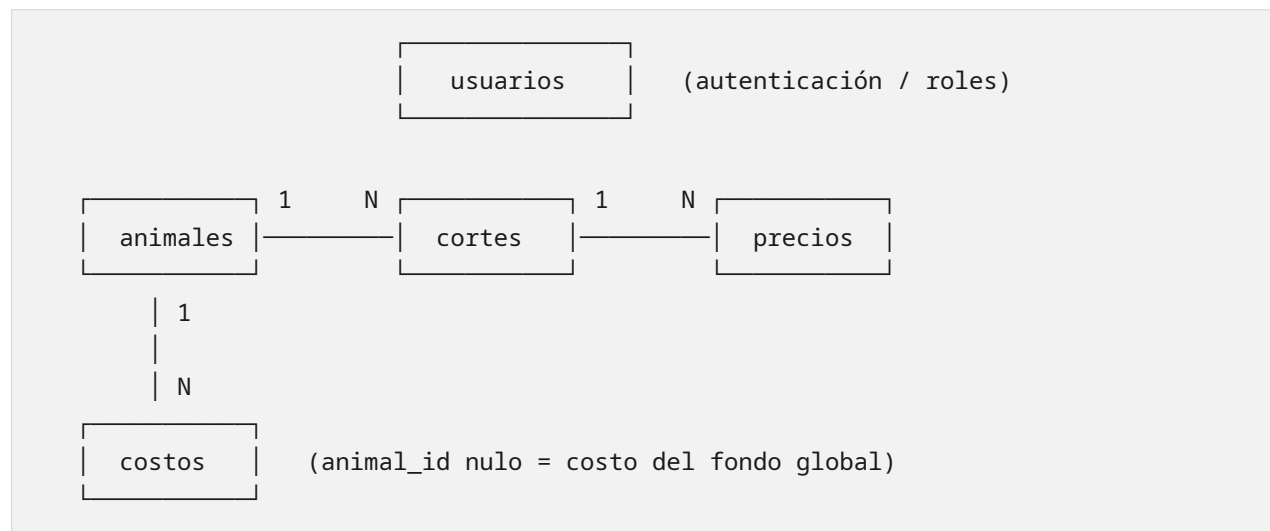
4.5 Requisitos de software y hardware

Requisito	Mínimo recomendado
Sistema operativo	Linux, macOS o Windows 10/11
Python	3.11 o superior
Node.js	18 o superior
Memoria RAM	4 GB (8 GB recomendado para desarrollo)
Conectividad	Acceso a internet (Supabase y servicio DANE-SIPSA)
Navegador	Chrome, Edge o Firefox en versión reciente

5. Modelo de Datos

5.1 Diagrama entidad-relación

El modelo se compone de cinco entidades. Un animal agrupa varios cortes y varios costos; cada corte mantiene un histórico de precios calculados. Los usuarios son una entidad independiente dedicada a la autenticación. Los datos de mercado SIPSA no se almacenan: se consultan en vivo al servicio del DANE cuando se necesitan.



Relaciones definidas con SQLAlchemy:

- animales → cortes: uno a muchos, con borrado en cascada.
- animales → costos: uno a muchos, con borrado en cascada (animal_id puede ser nulo para costos globales).
- cortes → precios: uno a muchos, con borrado en cascada (histórico de cálculos).

5.2 Detalle de tablas

Tabla: animales

Campo	Tipo	Restricciones / Descripción
id	Integer	PK, autoincremental.
codigo	String(50)	Único, obligatorio. Identificador del animal.
tipo	String(20)	Obligatorio. Por defecto «BOVINO».
raza	String(100)	Opcional.
peso_vivo	Float	Obligatorio, > 0. Peso en pie.

peso_canal	Float	Opcional al registrar; necesario para calcular precios.
rendimiento_canal	Float	Calculado: $(\text{peso_canal} / \text{peso_vivo}) \times 100$. Nulo si falta el peso canal.
calidad	String(20)	Por defecto «PRIMERA».
precio_compra	Float	Obligatorio, > 0. Costo de adquisición.
fecha_sacrificio	DateTime	Opcional.
fecha_registro	DateTime	Automática al crear.
notas	String(500)	Opcional.

Tabla: cortes

Campo	Tipo	Restricciones / Descripción
id	Integer	PK.
animal_id	Integer	FK → animales.id. Obligatorio.
nombre	String(100)	Obligatorio. Nombre del corte.
categoria	String(50)	PREMIUM / ESTANDAR / ECONOMICO. Por defecto «ESTANDAR».
peso_kg	Float	Obligatorio. Peso del corte.
factor_complejidad	Float	Intensidad de proceso del corte (ponderador ABC). Se asigna automáticamente según el nombre; 1,0 = corte promedio.
costo_unitario	Float	Costo ABC por kg, escrito en el último cálculo de precios.
precio_sugerido	Float	Resultado del cálculo de precios.
precio_mercado_sipsa	Float	Precio de referencia SIPSA aplicado.
margen_ganancia	Float	Margen real resultante.
activo	Boolean	Por defecto verdadero.
fecha_registro	DateTime	Automática al crear.

Tabla: costos

Campo	Tipo	Restricciones / Descripción
id	Integer	PK.
animal_id	Integer	FK → animales.id. Nulo = costo global.
concepto	String(200)	Obligatorio. Ej.: transporte, sacrificio.
categoria	String(100)	Opcional.
valor	Float	Obligatorio. Monto del costo en pesos.
unidad	String(50)	Por defecto «por_animal».
inductor	String(30)	Inductor ABC: KG, HORAS_HOMBRE, KWH, M3_REFRIG o FIJO. Por defecto «KG».
fecha_registro	DateTime	Automática al crear.
notas	String(500)	Opcional.

Tabla: precios (histórico de cálculos)

Campo	Tipo	Restricciones / Descripción
id	Integer	PK.
corte_id	Integer	FK → cortes.id. Obligatorio.
precio_costo_unitario	Float	Costo ABC por kg empleado en el cálculo.
precio_sugerido	Float	Precio de venta sugerido.
margen_objetivo	Float	Margen objetivo solicitado (por defecto 25 %).
precio_sipsa_referencia	Float	Precio de mercado usado como referencia.
precio_minimo_viable	Float	Precio mínimo para alcanzar el margen.
precio_maximo_mercado	Float	Precio máximo de mercado observado.
fecha_calculo	DateTime	Automática al calcular.

activo	Boolean	Por defecto verdadero.
--------	---------	------------------------

Tabla: usuarios

Campo	Tipo	Restricciones / Descripción
id	Integer	PK.
nombre	String(100)	Obligatorio.
email	String(150)	Único, obligatorio.
username	String(50)	Único, obligatorio.
hashed_password	String(255)	Contraseña con hash bcrypt. Nunca en texto plano.
rol	Enum	ADMIN o ESTANDAR.
activo	Boolean	Habilitación de la cuenta.
password_version	Integer	Versiona la contraseña para invalidar tokens previos.
fecha_creacion	DateTime	Automática al crear.
ultimo_acceso	DateTime	Marca del último inicio de sesión.

6. Lógica de Negocio: Costeo ABC y Cálculo de Precios

El corazón funcional del sistema reside en el servicio de análisis (app/services/analisis_service.py) y en el procesador de datos de mercado (app/sipsa/processor.py). El cálculo combina dos ideas: primero, repartir el costo total del animal entre sus cortes de manera proporcional a lo que cada corte realmente consume (costeo ABC); segundo, contrastar ese costo con el precio del mercado mayorista para llegar a un precio sugerido razonable.

6.1 Rendimiento en canal

Si se dispone del peso canal, el rendimiento se calcula como $\text{rendimiento} = (\text{peso_canal} / \text{peso_vivo}) \times 100$. Cuando el peso canal no se ha registrado, el rendimiento queda nulo: es un dato real que se mide tras el sacrificio, y el sistema prefiere no inventar un valor por defecto.

6.2 Costo directo por kilogramo de canal

Este indicador, visible en el panel, agrega el precio de compra y los costos asignados al animal, dividido por el peso de la canal:

```
costo_adicional = Σ (valor de cada costo asignado al animal)

costo_por_kg = (precio_compra + costo_adicional) / peso_canal
```

Si el animal no tiene peso canal registrado, el indicador queda nulo. Conviene precisar que este es un promedio informativo del animal completo; el costo que se usa para fijar precios es el costo ABC por corte, que se describe a continuación.

6.3 Costeo basado en actividades (ABC)

Repartir todos los costos por peso trataría igual un kilogramo de lomo fino y uno de carne molida, cuando el primero exige bastante más trabajo de proceso. Para corregirlo, cada costo registrado declara un inductor que determina cómo se distribuye entre los cortes del animal:

Inductor	Se reparte por	Ejemplo de costo
KG	Peso del corte	Transporte
FIJO	Peso del corte	Tarifa de sacrificio
HORAS_HOMBRE	Peso × factor de complejidad	Mano de obra de desposte
KWH	Peso × factor de complejidad	Energía de refrigeración
M3_REFRIG	Peso × factor de complejidad	Espacio en cuartos fríos

El factor de complejidad refleja cuánto trabajo exige cada corte respecto al promedio y se asigna automáticamente a partir del nombre, para que el usuario no tenga que estimar un número:

Corte	Factor	Corte	Factor
Lomo fino	1,5	Bola negra / Lagarto	1,1
Punta de anca	1,3	Costilla / Brazo	1,0
Lomo de aguja / Cadera / Muchacho	1,2	Pecho	0,9
Corte no reconocido	1,0	Molida	0,8

Con esos elementos, la distribución se realiza así:

```

peso_total = Σ peso de los cortes del animal
base_total = Σ (peso × factor_complejidad)

# Participan los costos del animal y los del fondo global (animal_id nulo)
costo_por_peso    = Σ costos con inductor KG o FIJO
costo_por_proceso = Σ costos con inductor HORAS_HOMBRE, KWH o M3_REFRIG

para cada corte:
    parte_peso    = peso / peso_total
    parte_proceso = (peso × factor_complejidad) / base_total

    costo_material    = precio_compra × parte_peso
    costo_actividades = costo_por_peso × parte_peso
                      + costo_por_proceso × parte_proceso

    costo_unitario = (costo_material + costo_actividades) / peso

```

Un detalle importante: el precio de compra se reparte solo entre los cortes vendibles registrados. Como la suma de cortes pesa menos que la canal (huesos, recortes, merma), el costo de esa diferencia queda absorbido por los cortes aprovechables, que es lo que ocurre en la práctica comercial. Por la misma razón, el costo ABC por kilogramo resulta algo mayor que el costo directo por kilogramo de canal de la sección 6.2.

6.4 Generación del precio sugerido

Para cada corte, el costo unitario ABC se transforma en precio sugerido en cuatro pasos:

```

# 1. Saneamiento del margen (si margen ≥ 100 %, se normaliza a 30 %)

```

```

precio_base = costo_unitario / (1 - margen_objetivo / 100)

# 2. Ajuste por categoría del corte
factor = { PREMIUM: 1.15, ESTANDAR: 1.00, ECONOMICO: 0.88 }
precio_ajustado = precio_base × factor[categoria]

# 3. Mezcla con el precio de mercado SIPSA (si está disponible)
si hay precio_sipsa:
    precio_final = precio_ajustado × 0.6 + precio_sipsa × 0.4
si no:
    precio_final = precio_ajustado

# 4. Margen real resultante
margen_real = (precio_final - costo_unitario) / precio_final × 100

```

Interpretación de cada paso:

- **Precio base:** Es el precio mínimo que recupera el costo y alcanza el margen objetivo; se reporta como «precio mínimo viable».
- **Factor de categoría:** Premia los cortes premium (+15 %) y abarata los económicos (–12 %), reflejando su valor comercial relativo.
- **Mezcla con SIPSA:** Pondera 60 % el precio basado en costos y 40 % el precio de mercado, para que la sugerencia no se aleje de la realidad del mercado mayorista. Si no hay referencia para ese corte, el precio queda determinado solo por costos.
- **Margen real:** Mide el margen efectivo del precio final; si la mezcla con el mercado lo dejó por debajo del objetivo, la interfaz lo resalta para que el usuario lo revise.

El cálculo exige que el animal tenga peso canal registrado; en caso contrario la API responde con un error 400 indicándolo. Cada ejecución actualiza los campos del corte (costo unitario, precio sugerido, referencia de mercado y margen) e inserta un registro en la tabla precios, conservando la trazabilidad de cada cálculo.

6.5 Ejemplo numérico

Animal comprado en \$2.500.000, con peso vivo de 450 kg y peso canal de 250 kg (rendimiento 55,6 %). Costos registrados: transporte \$150.000 (KG), sacrificio \$120.000 (FIJO), energía de frío \$80.000 (KWH) y mano de obra \$200.000 (HORAS_HOMBRE). Para simplificar, el despiece se reduce a tres cortes que suman 220 kg vendibles:

Corte	Peso	Factor	Costo material	Costo actividades	Costo unitario
Lomo fino (PREMIUM)	10 kg	1,5	\$113.636	\$32.563	\$14.620/kg
Cadera	60 kg	1,2	\$681.818	\$171.027	\$14.214/kg

(ESTANDAR)					
Molida (ECONOMICO)	150 kg	0,8	\$1.704.545	\$346.410	\$13.673/kg

La suma de costos distribuidos (\$3.050.000) coincide con el total real (compra más costos), y el lomo fino termina con el costo unitario más alto pese a ser el corte más liviano, porque concentra más trabajo de proceso. Con ese costo, el precio de la cadera (margen objetivo del 25 % y referencia SIPSA de \$22.000/kg) se obtiene así:

Paso	Operación	Resultado
Precio base (mínimo viable)	$14214 / (1 - 0.25)$	\$18.952
Ajuste categoría (×1,00)	18952×1.00	\$18.952
Mezcla SIPSA	$18952 \times 0.6 + 22000 \times 0.4$	\$20.171
Margen real	$(20171 - 14214) / 20171 \times 100$	≈ 29,5 %

7. Integración con DANE-SIPSA

7.1 Qué es SIPSA

SIPSA es el Sistema de Información de Precios y Abastecimiento del Sector Agropecuario, administrado por el DANE. Publica precios mayoristas de productos agropecuarios —entre ellos la carne bovina— en distintas centrales de abasto del país. DesposteApp consume este servicio para anclar sus precios sugeridos a referencias oficiales de mercado.

7.2 Cliente SOAP

El cliente (`app/sipsa/client.py`) emplea la librería `zeep` para invocar el servicio web SOAP del DANE. Filtra los productos bovinos mediante un conjunto de palabras clave (`res`, `bovina`, `lomo`, `costilla`, `cadera`, etc.) y normaliza cada registro a una estructura común: producto, ciudad, mercado, precios (promedio, mínimo, máximo), unidad y fechas.

7.3 Estrategia de tolerancia a fallos (4 niveles)

La obtención de datos está diseñada para no fallar de cara al usuario. Se intentan, en orden, distintos métodos del servicio y, como último recurso, un conjunto de datos de demostración:

Nivel	Fuente	Método / contenido
1	DANE — por ciudad	<code>promediosSipsaCiudad()</code>
2	DANE — parcial	<code>promediosSipsaParcial()</code> (min/máx por ubicación)
3	DANE — semanal mayorista	<code>promediosSipsaSemanalMadr()</code>
4	Datos de demostración	12 cortes × 4 mercados × 12 semanas, precios realistas Colombia

Consulta en vivo, sin persistencia

Los datos de mercado se obtienen y procesan en memoria cada vez que se necesitan (consulta del módulo SIPSA, panel o cálculo de precios) y no se guardan en la base de datos. Esto simplifica el sistema y evita acumular duplicados, a cambio de no construir un histórico propio de precios; esa decisión se discute en la sección de limitaciones.

7.4 Procesamiento y limpieza de datos

El procesador (`app/sipsa/processor.py`) aplica una secuencia de limpieza estadística sobre los datos recibidos:

- **Saneamiento:** Conversión de precios a numérico y descarte de valores nulos o no positivos.
- **Outliers:** Eliminación de valores atípicos mediante el rango intercuartílico (regla $1,5 \times IQR$).
- **Agregación:** Agrupación por corte (mediante un mapa de palabras clave con normalización de tildes y mayúsculas) y cálculo de promedio, mínimo, máximo y desviación estándar.

7.5 Cálculo de la tendencia

La columna de tendencia que se muestra en el panel y en el módulo SIPSA (SUBIENDO, BAJANDO o ESTABLE) se calcula por corte de la siguiente manera: se ordenan las observaciones por fecha, se toman las cuatro más recientes y se compara el precio de la primera con el de la última. Si la variación porcentual supera el +3 % la tendencia es SUBIENDO; si cae por debajo del -3 %, BAJANDO; en la franja intermedia, ESTABLE. Cuando hay menos de tres observaciones con fecha, el sistema devuelve ESTABLE por defecto. Es deliberadamente un método simple — compara extremos, no ajusta una regresión—, suficiente como señal visual rápida pero no como análisis estadístico de series de tiempo.

8. Referencia de la API REST

Todos los endpoints se exponen bajo el prefijo `/api/v1`. Salvo el inicio de sesión, todas las rutas requieren autenticación. La documentación interactiva (Swagger UI) está disponible en `/docs` únicamente cuando la variable `DEBUG` está activada.

8.1 Autenticación y usuarios — `/api/v1/auth`

Método	Ruta	Descripción	Acceso
POST	<code>/auth/login</code>	Inicia sesión y emite la cookie JWT.	Público (5/min)
POST	<code>/auth/logout</code>	Cierra sesión y elimina la cookie.	Autenticado
GET	<code>/auth/me</code>	Devuelve los datos del usuario actual.	Autenticado
POST	<code>/auth/cambiar-password</code>	Cambia la contraseña propia (invalida tokens previos).	Autenticado
GET	<code>/auth/usuarios</code>	Lista los usuarios.	ADMIN
POST	<code>/auth/usuarios</code>	Crea un usuario.	ADMIN
PUT	<code>/auth/usuarios/{id}</code>	Actualiza nombre, correo, rol o estado; opcionalmente restablece la contraseña (mínimo 8 caracteres, invalida las sesiones del usuario).	ADMIN
DELETE	<code>/auth/usuarios/{id}</code>	Elimina un usuario.	ADMIN

8.2 Animales — `/api/v1/animales`

Método	Ruta	Descripción
POST	<code>/animales/</code>	Crea un animal (calcula el rendimiento en canal si hay datos).
GET	<code>/animales/</code>	Lista animales (paginado: skip, limit).
GET	<code>/animales/{id}</code>	Obtiene un animal por su identificador.

PUT	/animales/{id}	Actualiza un animal y recalcula su rendimiento.
DELETE	/animales/{id}	Elimina un animal (y sus cortes/costos en cascada).

8.3 Cortes — /api/v1/cortes

Método	Ruta	Descripción
POST	/cortes/	Crea un corte (asigna automáticamente el factor de complejidad).
GET	/cortes/	Lista cortes (paginado).
GET	/cortes/animal/{id}	Lista los cortes de un animal.
GET	/cortes/{id}	Obtiene un corte.
PUT	/cortes/{id}	Actualiza un corte.
DELETE	/cortes/{id}	Elimina un corte.
GET	/cortes/{id}/precios	Histórico de precios calculados del corte.

8.4 Costos — /api/v1/costos

Método	Ruta	Descripción
POST	/costos/	Crea un costo (por animal o global) con su inductor ABC.
GET	/costos/	Lista costos (paginado).
GET	/costos/animal/{id}	Lista los costos de un animal.
GET	/costos/{id}	Obtiene un costo.
PUT	/costos/{id}	Actualiza un costo.
DELETE	/costos/{id}	Elimina un costo.

8.5 SIPSA — /api/v1/sipsa

Método	Ruta	Descripción
GET	/sipsa/consultar	Consulta en vivo los precios de

		mercado (parámetros de fecha opcionales; devuelve hasta 200 registros).
GET	/sipsa/promedios	Promedios, rangos, desviación y tendencia por corte, calculados sobre la consulta en vivo.

8.6 Análisis — /api/v1/analysis

Método	Ruta	Descripción
GET	/analysis/dashboard	Indicadores globales (totales, costo y margen promedio).
POST	/analysis/calcular-precios/{id}	Calcula y persiste los precios de los cortes de un animal (parámetro margen; exige peso canal).
GET	/analysis/costo-kg/{id}	Devuelve el costo directo por kilogramo de un animal.

9. Seguridad

9.1 Autenticación basada en JWT

El inicio de sesión válido genera un JSON Web Token firmado con el algoritmo HS256. El token incluye el usuario (sub), el rol, el identificador y la versión de contraseña (pwd_v), con una expiración configurable mediante la variable `TOKEN_EXPIRE_MINUTES` (60 minutos por defecto).

9.2 Cookie HttpOnly

El token se entrega al navegador en una cookie `HttpOnly` con `SameSite=Lax`. Al no ser accesible desde JavaScript, se mitiga el robo de token mediante ataques XSS. En producción (DEBUG desactivado) la cookie se marca como `Secure`, exigiendo HTTPS. Para clientes de API y para la documentación interactiva se acepta también el token en la cabecera `Authorization (Bearer)`.

9.3 Hash de contraseñas

Las contraseñas nunca se almacenan en texto plano: se guardan con hash `bcrypt` y sal aleatoria. La verificación se realiza comparando el hash, sin exponer jamás la contraseña original.

9.4 Invalidación de sesiones por versión de contraseña

Cada usuario tiene un campo `password_version` que viaja dentro del token. Cuando la contraseña cambia —sea porque el propio usuario la actualiza desde su perfil o porque un administrador la restablece desde la gestión de usuarios—, ese número se incrementa y todos los tokens emitidos antes del cambio dejan de ser aceptados de inmediato, aunque no hayan expirado.

9.5 Autorización por roles

La dependencia `get_current_user` valida el token en cada petición protegida y verifica que la cuenta siga activa; la dependencia `require_admin` restringe las rutas de administración de usuarios al rol `ADMIN`, devolviendo 403 en caso contrario.

9.6 Limitación de tasa (rate limiting)

El endpoint de inicio de sesión está limitado a 5 intentos por minuto y dirección IP mediante `SlowAPI`, mitigando ataques de fuerza bruta.

9.7 CORS

La política de CORS del backend permite únicamente los orígenes de desarrollo del frontend (`localhost:5173` y `localhost:3000`), con credenciales habilitadas y métodos y cabeceras acotados.

9.8 Gestión de secretos

La clave de firma (SECRET_KEY), la cadena de conexión a la base de datos y las credenciales de los usuarios iniciales se leen exclusivamente de variables de entorno. El archivo .env está excluido del control de versiones (.gitignore). Si falta la SECRET_KEY, el backend se niega a arrancar en lugar de usar una clave por defecto insegura.

Recomendaciones para producción

Para un despliegue productivo se recomienda: rotar la SECRET_KEY y mantenerla en un gestor de secretos; forzar HTTPS extremo a extremo; restringir CORS al dominio real; aplicar políticas de contraseña robustas; añadir registro de auditoría; y revisar las versiones de dependencias frente a vulnerabilidades conocidas.

10. Frontend (Interfaz de Usuario)

10.1 Tecnología y organización

La interfaz es una SPA construida con React 18 y Vite. El estado de sesión se gestiona con un contexto global (AuthContext), el enrutamiento con React Router 6 y la comunicación con la API mediante una instancia centralizada de Axios (services/api.js). La iconografía utiliza lucide-react, una librería de iconos SVG, en lugar de caracteres decorativos, para mantener una apariencia uniforme en cualquier sistema operativo.

10.2 Gestión de la sesión en el cliente

Como el token vive en una cookie HttpOnly, el JavaScript del navegador no puede leerlo. El frontend guarda en localStorage únicamente una marca de sesión activa y el perfil básico del usuario (nombre, usuario y rol) para pintar la interfaz. Al recargar la página, si existe la marca de sesión, el contexto consulta /auth/me para validar que la cookie siga vigente y recuperar el perfil; si la respuesta es 401, limpia el estado y redirige al inicio de sesión.

10.3 Páginas y rutas

Ruta	Página	Descripción	Acceso
/login	LoginPage	Inicio de sesión.	Público
/	DashboardPage	Panel de indicadores y tabla de referencia SIPSA.	Autenticado
/animales	AnimalesPage	Gestión de animales.	Autenticado
/cortes	CortesPage	Gestión de cortes (muestra factor ABC y costos calculados).	Autenticado
/costos	CostosPage	Gestión de costos con inductor ABC.	Autenticado
/sipsa	SIPSAPage	Consulta de precios de mercado (promedios y detalle).	Autenticado
/analisis	AnalisisPage	Cálculo de precios y análisis por animal.	Autenticado
/usuarios	UsuariosPage	Administración de usuarios y restablecimiento de contraseñas.	ADMIN
/perfil	PerfilPage	Perfil y cambio de	Autenticado

		contraseña propia.	
--	--	--------------------	--

10.4 Rutas protegidas

El componente `ProtectedRoute` envuelve las vistas autenticadas y redirige a `/login` cuando no hay sesión. Las rutas marcadas como `adminOnly` (p. ej. `/usuarios`) exigen además el rol `ADMIN`. El interceptor de `Axios` fuerza la redirección al recibir una respuesta 401.

10.5 Componentes reutilizables

- **Navbar:** Barra de navegación principal con accesos según el rol.
- **ProtectedRoute:** Control de acceso a las vistas autenticadas y de administrador.
- **Toast:** Notificaciones emergentes de éxito o error.
- **KpiCard / StatsCard:** Tarjetas de indicadores para el panel.

11. Instalación y Despliegue

11.1 Requisitos previos

- Python 3.11 o superior.
- Node.js 18 o superior.
- Una base de datos PostgreSQL accesible (p. ej. un proyecto en Supabase).

11.2 Configuración de la base de datos (Supabase)

1. Crear un proyecto en Supabase.
2. En Settings → Database → Connection string, copiar la URL del «Session Pooler» (puerto 5432).
3. Registrar dicha URL en la variable DATABASE_URL del archivo backend/.env.

Las tablas y los usuarios iniciales se crean automáticamente al arrancar el backend.

11.3 Backend

```
cd backend
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8001

# Documentación interactiva (Swagger, solo con DEBUG=true):
#   http://localhost:8001/docs
```

11.4 Frontend

```
cd frontend
npm install
npm run dev          # servidor de desarrollo en http://localhost:5173
npm run build        # build de producción
```

Coherencia de puertos

El proxy del frontend (frontend/vite.config.js) apunta al backend en el puerto 8001. Si se ejecuta el backend en otro puerto, debe actualizarse el destino del proxy para que coincida.

11.5 Variables de entorno (backend/.env)

Variable	Obligatoria	Descripción
DATABASE_URL	Sí	Cadena de conexión PostgreSQL (Supabase Session Pooler).
SECRET_KEY	Sí	Clave de firma de los JWT. Sin ella, el backend no arranca.
DEBUG	No	true habilita /docs y cookies no seguras (desarrollo).
TOKEN_EXPIRE_MINUTES	No	Expiración del token en minutos (por defecto 60).
SIPSA_WSDL	No	URL del servicio SOAP del DANE.
ADMIN_USERNAME / _PASSWORD / _EMAIL	No*	Credenciales del usuario administrador inicial.
OPERADOR_USERNAME / _PASSWORD / _EMAIL	No*	Credenciales del usuario operador inicial.

* Si no se define la contraseña de un usuario inicial, dicho usuario simplemente no se crea.

12. Operación y Mantenimiento

12.1 Arranque del sistema

Al iniciar, el backend: (1) crea el esquema de base de datos si no existe; (2) aplica de forma idempotente las columnas del modelo ABC (inductor en costos; factor de complejidad y costo unitario en cortes) sobre bases de datos creadas con versiones anteriores; (3) crea los usuarios iniciales definidos por entorno; y (4) registra los manejadores de límite de tasa y CORS. El frontend se sirve por separado y se comunica con el backend a través del proxy.

12.2 Consideraciones de mantenimiento

- **Migraciones:** El esquema se crea con `create_all` y unos `ALTER TABLE` idempotentes; para cambios de modelo en producción se recomienda adoptar una herramienta de migraciones (p. ej. Alembic).
- **Respaldos:** Se aconseja programar respaldos periódicos de la base de datos (gestionados por Supabase).
- **Dependencias:** Conviene revisar periódicamente las versiones de las dependencias y aplicar parches de seguridad.
- **Registro:** El backend registra eventos relevantes (consultas SIPSA, activación del fallback, errores) mediante el módulo de logging.

13. Limitaciones y Alcance No Cubierto

DesposteApp es un prototipo académico y conviene ser explícitos sobre sus fronteras. Esta sección reúne lo que el sistema no hace, las simplificaciones que asume y las decisiones de diseño cuyas consecuencias deben conocerse antes de usarlo o extenderlo. Nada de lo que sigue impide el funcionamiento del prototipo; son los puntos donde un uso en producción exigiría trabajo adicional.

13.1 Alcance funcional no cubierto

- El sistema termina en la sugerencia de precio: no gestiona inventarios, ventas, facturación, clientes ni proveedores.
- No genera reportes exportables (PDF o Excel); los resultados se consultan en pantalla.
- El análisis de precios se realiza animal por animal; no existe un cálculo por lotes ni una vista consolidada por periodo (semana, mes).
- La merma no se modela como entidad propia: la diferencia entre el peso de la canal y la suma de los cortes registrados se absorbe implícitamente en el costo de material de los cortes vendibles. Es un supuesto razonable, pero el sistema no informa cuánta merma hubo ni permite valorarla por separado (p. ej. venta de hueso o sebo).
- Maneja una sola moneda (pesos colombianos) y no contempla impuestos, descuentos ni promociones en el precio sugerido.
- No construye un histórico propio de precios de mercado: los datos SIPSA se consultan en vivo y se descartan tras usarse. Por eso no es posible graficar la evolución de un precio a lo largo de meses, y la tendencia mostrada depende de las observaciones que el servicio del DANE devuelva en esa consulta puntual.
- La interfaz está disponible solo en español y pensada para pantallas de escritorio; no se ha hecho una adaptación seria a dispositivos móviles ni una auditoría de accesibilidad.
- Está pensado para un solo negocio (una planta de desposte); no es multiempresa ni multi-sucursal.

13.2 Limitaciones del modelo de cálculo

El modelo de precios funciona y es trazable, pero varios de sus parámetros son heurísticos: se fijaron por criterio propio durante el desarrollo y no fueron calibrados contra datos reales de ventas. En concreto:

- Los factores de categoría (+15 % premium, -12 % económico), la ponderación de la mezcla con el mercado (60/40) y los factores de complejidad (0,8 a 1,5) son constantes definidas en el código; no son configurables desde la interfaz ni se estimaron empíricamente.
- El costeo ABC reparte montos en pesos según el inductor declarado; no calcula tarifas a partir de cantidades medidas (horas-hombre reales, kWh del medidor). Es decir, el usuario

registra «mano de obra: \$200.000» y el sistema lo distribuye; no registra «8 horas a \$25.000».

- La tendencia de mercado compara el primer y el último punto de las cuatro observaciones más recientes, con un umbral fijo de ± 3 %. Un valor atípico en cualquiera de los dos extremos puede cambiar la etiqueta; no se ajusta una regresión ni se suaviza la serie.
- El costo directo por kilogramo del panel considera solo los costos asignados al animal, mientras que el costeo ABC incluye también los costos del fondo global. Ambos indicadores son coherentes por separado, pero no son directamente comparables entre sí.
- El cálculo exige el peso de la canal; si no se ha registrado, el sistema no estima un rendimiento teórico, simplemente no calcula. Es una decisión deliberada (no inventar datos), pero implica que el flujo se bloquea hasta tener la pesa real.

13.3 Limitaciones de los datos de mercado

- La integración depende de la disponibilidad del servicio SOAP del DANE, que en la práctica es intermitente. Cuando no responde, el sistema pasa a datos de demostración generados con una semilla fija: son verosímiles (rangos reales de Colombia 2024-2025) y estables entre ejecuciones, pero no son precios reales. La interfaz no distingue visualmente cuál de las dos fuentes está activa, de modo que el usuario debe ser consciente de esta posibilidad antes de tomar decisiones con esos valores.
- Algunos métodos del servicio solo entregan el precio promedio; en esos casos el mínimo y el máximo se estiman como ± 7 % del promedio, igual que en los datos de demostración.
- El filtrado de productos bovinos se hace por palabras clave sobre el nombre del producto; puede dejar pasar productos con nombres parecidos o excluir denominaciones regionales que no estén en el mapa de cortes.

13.4 Limitaciones técnicas y de seguridad

- No hay pruebas automatizadas (unitarias ni de integración) ni un pipeline de integración continua; la verificación ha sido manual.
- El esquema se crea con `create_all` más unos `ALTER TABLE` idempotentes; no hay una herramienta de migraciones real, por lo que la evolución del modelo en una base con datos exige cuidado.
- La limitación de tasa solo cubre el inicio de sesión y se mantiene en memoria: se reinicia con el proceso y no se comparte entre réplicas.
- No hay refresh tokens (la sesión expira y hay que volver a entrar), ni recuperación de contraseña por correo, ni autenticación de dos factores, ni registro de auditoría de acciones.
- CORS está fijado a los orígenes de desarrollo (localhost); un despliegue real requiere ajustarlo, junto con HTTPS extremo a extremo.
- El acceso a la base de datos es síncrono y está dimensionado para los volúmenes bajos de un entorno académico; no se ha hecho ninguna prueba de carga.

- Solo se documenta la ejecución en modo desarrollo; no hay imagen de contenedor (Docker) ni guía de despliegue productivo.

13.5 Trabajo futuro sugerido

De las limitaciones anteriores se desprende una lista natural de mejoras, ordenada por el valor que aportaría a la operación:

1. Persistir un histórico propio de precios SIPSA (con deduplicación por producto y semana) para graficar tendencias reales y mejorar el cálculo de tendencia con series de tiempo.
2. Indicar en la interfaz si los datos de mercado provienen del DANE o del conjunto de demostración.
3. Incorporar pruebas unitarias y de integración, y un pipeline de despliegue.
4. Adoptar migraciones de base de datos (Alembic).
5. Exportación de reportes (PDF/Excel) de precios y márgenes.
6. Hacer configurables los parámetros del modelo (factores de categoría, mezcla con mercado, factores de complejidad) y, a futuro, calibrarlos con datos reales de ventas.
7. Modelar la merma y los subproductos del desposte como parte del cálculo.
8. Análisis consolidado por lotes y por periodo.

— Fin del documento —